

# When Minimax Hurts: A Pre-Registered Study of Robust Policy Optimization under Tool Drift

Anonymous Authors

## Abstract

Tool-agent recovery methods combine several changes at once: additional failure-conditioned rollouts, different advantage groups, recovery-specific rewards, and worst-group reweighting. A gain under such a bundle does not identify which recovery mechanism caused it. We conduct a controlled audit of Minimax Recovery Policy Optimization (MRPO) for a Qwen3.5-9B LoRA agent in executable, stateful tool environments. Before training, we calibrate task difficulty, freeze task-fault manifests and state-based verifiers, and preregister three seed-level directional hypotheses. We then compare MRPO with GRPO, no-fission, and no-minimax arms while matching primary rollout count and advantage grouping, fixing the branch budget for fission-bearing arms, and pairing evaluation episodes. Across seeds 23, 29, and 43, none of the three hypothesized mechanism advantages replicates: the pooled paired success differences are 0/96 for MRPO versus GRPO (95% bootstrap CI  $[-0.0938, 0.0938]$ ),  $-2/96$  versus no-fission ( $[-0.1042, 0.0625]$ ), and  $-14/96$  versus no-minimax ( $[-0.2604, -0.0312]$ ). The preregistered regression gate passes, so the result is not explained by a catastrophic seed, but minimax weighting is consistently harmful in the primary suite. These findings do not reproduce or contradict the full Fission-GRPO system; they show that, in our controlled regime, branching and minimax mechanisms do not retain an identifiable advantage after estimator matching. The main contribution is therefore an auditable protocol and a multi-seed negative result that narrows when structured recovery objectives should be expected to help.

## 1 Introduction

Language models increasingly act through external tools, modify persistent state, and operate over multiple turns. Systems such as ReAct and ToolLLM established that models can interleave reasoning with actions and learn broad tool interfaces [10, 19]. Stateful benchmarks and reinforcement-learning frameworks now evaluate complete goals rather than isolated function selection [1, 2, 6, 17, 20]. Yet tool execution in deployment is not stable: authentication expires, timeouts obscure whether writes committed, responses become stale or truncated, and schemas drift. These events change observations and state transitions during an episode, so recovery requires more than retrying a failed call.

Recent methods respond with noise curricula, failure-conditioned rollout generation, and explicit budget allocation [3, 5, 21]. Such systems are necessarily composite. Failure branching changes the number and distribution of rollouts; recovery branches may use a different advantage group; extra turns change generated-token and tool-call budgets; and worst-group weighting changes the optimization target. Consequently, an end-to-end improvement does not by itself establish that branching or minimax weighting is the active mechanism. This is particularly important for tool-agent RL, where small frozen evaluation sets and floor or ceiling effects can make a single seed look decisive.

We study this identification problem through MRPO, a controlled recovery training system with typed failure-prefix branching, executable progress rewards, a bounded recovery budget, and fault-group minimax weighting. Our question is not whether this collection can be made to produce an isolated positive run. It is whether its proposed mechanisms retain a reproducible behavioral advantage once the rollout estimator and interaction budget are matched.

The evaluation is frozen before the formal comparison. We first calibrate a base adapter into a discriminative acceptance band, then freeze task-fault manifests, executable state verifiers, arm definitions, and stopping rules. Every arm is evaluated on the same episodes, enabling task-paired inference. The formal round uses Qwen3.5-9B with LoRA and exactly the preregistered seeds 23, 29, and 43. Three directional

hypotheses isolate MRPO versus GRPO, no-minimax, and no-fission. A hypothesis requires the same sign at all three seeds and a pooled paired-bootstrap confidence interval excluding zero; a separate regression gate can veto the round.

The result is informative precisely because it is negative. On the primary 32-episode suite, MRPO scores 16, 15, and 17 successes across the three seeds. Its paired differences versus GRPO are  $-1, +1, 0$ ; versus no-fission,  $0, -3, +1$ ; and versus no-minimax,  $-5, -6, -3$ . Thus no hypothesized advantage replicates. Moreover, the pooled interval for MRPO minus no-minimax lies below zero, identifying minimax weighting as harmful in this regime. The regression gate passes, which rules out a catastrophic-run interpretation.

This work makes four contributions:

- We provide a pre-calibrated, frozen evaluation protocol for executable tool-recovery RL, with task-paired outcomes, explicit regression gates, and directional multi-seed decision rules.
- We construct estimator-matched ablations that equalize primary rollout count and advantage grouping, with a fixed recovery budget for fission-bearing arms, separating mechanism claims from extra-sampling effects.
- We report a complete three-seed negative result: neither the full MRPO arm nor failure-prefix fission has a reproducible primary-suite advantage, while minimax weighting is consistently detrimental.
- We document the boundary of the evidence. The result concerns a controlled Qwen3.5-9B LoRA regime and is not an official reproduction of Fission-GRPO. Planned zero-touch external evaluation could not be run under a clearly applicable data and software license, so no external transfer claim is made.

The contribution is therefore not a new state of the art. It is evidence about identifiability: once estimators and budgets are aligned, intuitively appealing recovery components may have no measurable benefit, and robust worst-group optimization may even reduce task success.

## 2 Related Work

### 2.1 Tool-Using Language Agents and Agentic Reinforcement Learning

ReAct demonstrated that interleaving language reasoning and actions can improve interactive decision making [19], while ToolLLM scaled instruction tuning to a large collection of real-world APIs [10]. Stateful benchmarks subsequently made it possible to evaluate whether an agent completes a goal rather than merely predicts a function name.  $\tau$ -bench evaluates tool-agent-user interaction in realistic domains, and  $\tau^2$ -Bench extends the setting to dual-control conversations in which both the user and the agent affect the trajectory [1, 20].

Reinforcement-learning approaches target limitations of supervised tool traces. ToolRL studies reward design for tool learning [9]; AgentGym-RL trains agents over long-horizon, multi-turn interactions [17]; and VerlTool and RLFactory provide infrastructure for multi-turn rollout and post-training [2, 6]. Many recent systems use group-relative policy optimization, popularized in open-model reasoning by DeepSeekMath [13]. Our work shares the executable, multi-turn training setting but studies whether recovery mechanisms remain identifiable after matching rollout estimators and interaction budgets.

### 2.2 Robustness, Noise, and Open-World Tool Use

Robustness studies show that function-call performance is sensitive to natural-language reformulations and changes in the available tool set [11]. AgentNoiseBench broadens this analysis to user-side and tool-side noise [16]. ToolBench-X organizes recoverable tool-environment unreliability into specification drift, invocation error, execution failure, output drift, and cross-source conflict, with executable multi-step tasks [14]. OpenAgent studies query, action, observation, and domain shifts in an open-world hierarchy and finds that static training remains brittle under environmental change [7]. Sim-to-real work further cautions that simulator success need not transfer to realistic tool execution [22].

NoisyAgent introduces a noise curriculum for perturbed environments [3], and EnvFactory studies executable environment synthesis and robust RL at scale [18]. These methods show that exposure to diverse

environments can improve robustness. Our result is a boundary condition: in a strong-policy regime, environment difficulty can be dominated by the distance between the required recovery path and the turn budget. Floor and ceiling effects can then hide or mimic algorithmic gains. We therefore pre-calibrate the baseline into a fixed acceptance band before freezing the discriminative suite.

### 2.3 Failure Recovery and Reliability Evaluation

Fission-GRPO, accepted to ACL 2026, directly trains recovery by converting execution errors into on-policy corrective instances [21]. It augments failed trajectories with diagnostic feedback from a fine-tuned error simulator and resamples recovery rollouts, reporting gains on BFCL v4 Multi-Turn,  $\tau$ -bench, and  $\tau^2$ -Bench. Our failure-prefix branching is motivated by the same on-policy principle but uses typed executable failures and state verifiers, applies an explicit recovery budget, and adds fault-group minimax weighting. Crucially, our estimator-matched ablation does not establish an advantage for branching: the MRPO versus no-fission comparison changes sign across seeds and its pooled paired interval contains zero. This is not an official reproduction of Fission-GRPO; it shows that the general branching mechanism does not automatically transfer to our controlled fault families and training regime.

Recent benchmarks make recovery itself the evaluation target. ToolMaze uses DAG-based task complexity and a two-by-two taxonomy of explicit versus implicit and transient versus permanent perturbations; it reports that implicit semantic failures reduce perturbation recovery rate by roughly 37%<sup>1</sup>. Reliability-Bench evaluates a reliability surface over repeated-execution consistency, semantic perturbation intensity, and controlled fault intensity, including timeouts, rate limits, partial responses, and schema drift<sup>2</sup>. These evaluation-oriented works align with our separation of unconditional task success, fault exposure, conditional recovery, and recovery cost. Our additional focus is training-time identifiability: equalizing the number of primary rollouts, the advantage grouping unit, and the recovery budget before attributing a behavioral change to a recovery mechanism.

Recoverability can also be summarized through regularity or risk measures. The ERR measure studies whether tool-augmented agents recover from failures [15], while ToolChain-CRC controls trajectory risk under retrieval and tool-use drift [8]. In our protocol, recovery requires verifier-confirmed task success and at least one non-faulted action after the final observed failure, avoiding credit for a hidden successful commit followed by immediate termination.

### 2.4 Rollout Allocation and Worst-Group Optimization

Tree-based agent training can spend most of its budget on redundant branches. InfoTree casts rollout selection under a fixed budget as an informativeness problem [5]. MRPO instead scores failure prefixes using estimated frequency, non-recovery probability, uncertainty, group weakness, and expected cost. Our experiments show why generated-token accounting alone is insufficient: primary rollout count and advantage grouping must also be matched to identify a mechanism effect.

Worst-group objectives are closely related to group distributionally robust optimization, which upweights high-loss groups to improve worst-case generalization [12]. We use fault families as groups and retain a clean-group floor. Group DRO itself is not a contribution. Moreover, the three-seed minimax ablation is consistently unfavorable: removing minimax weighting improves the primary-suite result, with a pooled paired interval below zero for MRPO minus no-minimax. We therefore present minimax weighting as a tested mechanism with a negative result, not as an empirically validated improvement.

**Positioning.** The nearest work contributes noisy-environment training, recovery-instance construction, and reliability evaluation. Our contribution is an auditable protocol for determining whether such mechanisms cause behavioral improvement: pre-calibrated and frozen executable tasks, estimator-matched ablations, multi-seed task-paired inference, and clean/fault regression gates. The protocol finds no reproducible MRPO advantage in the studied regime and identifies a stable negative effect from minimax weighting. This narrows the conditions under which structured recovery objectives should be expected to help, rather than claiming a new state of the art.

<sup>1</sup>ToolMaze: <https://arxiv.org/abs/2606.05806>.

<sup>2</sup>ReliabilityBench: <https://arxiv.org/abs/2601.06112>.

### 3 Problem Formulation and Audited Mechanisms

We model tool use as a family of constrained partially observable Markov decision processes indexed by fault group  $z \in \mathcal{Z}$ . An episode produces a verifier-defined binary task outcome and consumes interaction cost. MRPO combines four mechanisms: it branches from typed failures in on-policy trajectories, rewards verifier-observed recovery progress, assigns a bounded recovery budget across eligible prefixes, and reweights fault groups toward higher loss while retaining a clean-group floor.

Our empirical target is the incremental behavioral effect of those mechanisms. The formal comparison therefore includes GRPO, no-fission, and no-minimax arms. The matched arms share the base checkpoint, prompts, task distribution, primary rollout count, advantage grouping unit, optimizer schedule, and frozen evaluation episodes. The two fission-bearing arms also share the same recovery budget. No-fission tests whether the failure-prefix branches help once their estimator consequences are matched. No-minimax tests the marginal effect of worst-group reweighting. We treat MRPO as an audited composite rather than assuming that each component is beneficial by construction.

## 4 Experimental Setup

### 4.1 Frozen tasks and calibration

The controlled environment contains executable stateful workflows with typed fault interventions and exact final-state verifiers. The formal training set has 32 L1-geometry tasks: eight examples each for timeout, rate limiting, empty response, and truncated response. Every task uses five candidates, four storage bins, and a deficit of three. A separate 32-task held-out set contains authentication expiry, schema drift, delayed commit, and malformed redirect, again with eight examples per fault family. Training and evaluation task identities are disjoint.

Before any formal arm sees the primary suite, we calibrate the r10 reference adapter once. Its 15/32 success rate falls within the preregistered 30–70% acceptance band, so dsweep-L1 is frozen as the primary discriminative instrument. Task rows, fault assignments, split labels, turn budgets, and verifiers are not changed after this decision.

### 4.2 Model and training arms

All formal runs use Qwen3.5-9B, instantiated as `Qwen3_5ForConditionalGeneration`, with LoRA adapters initialized from r10. Training lasts 10 updates with batch size 32 and group size 4, giving 128 primary rollouts per update in every arm. Primary advantages are grouped by task identity. The optimizer uses learning rate  $5 \times 10^{-6}$ , KL coefficient 0.02, temperature 0.8, top- $p$  0.95, maximum gradient norm 1.0, and a 512-token generation cap. Each task permits 11 interaction turns.

We compare four arms. MRPO includes both failure-prefix fission and fault-group minimax weighting. GRPO includes neither mechanism. No-fission removes recovery branches while retaining the otherwise matched primary estimator. No-minimax retains fission but removes group reweighting. For the two fission-bearing arms, the additional recovery budget is fixed at 32 branch rollouts and 12,288 generated tokens per update. Thus all arms match primary sampling and task-level advantage grouping, while the extra branch cost remains an explicit property of fission rather than being hidden in the baseline.

### 4.3 Seeds and execution protocol

The pilot uses seed 23. Its preregistered continuation signal is satisfied, so the exact protocol is copied to seeds 29 and 43. These are the only formal seeds. Each of the 12 runs completes all 10 updates. Long-running jobs use unbuffered Python under process supervisors; a run is evaluated only after its final adapter is present and loadable.

| Seed | MRPO | GRPO | no-fission | no-minimax | r10 |
|------|------|------|------------|------------|-----|
| 23   | 16   | 17   | 16         | 21         | 15  |
| 29   | 15   | 14   | 18         | 21         | 15  |
| 43   | 17   | 17   | 16         | 20         | 15  |

Table 1: Success counts out of 32 on the frozen primary suite. The r10 column is the calibrated reference adapter, not a formal mechanism arm.

#### 4.4 Frozen evaluation

Each final adapter is evaluated on nine frozen suites: the dsweep-L1 primary and held-out instruments, five legacy suites, and two fault-v2 suites. The evaluation command does not override the row-level turn budget. The b2dev suite uses its development split; every other suite uses its frozen split. Decoding is greedy with evaluation seed 7 and a 512-token generation cap. All arms within a training seed receive identical task rows and fault assignments.

Success is defined only by the executable state verifier. The analysis reads episode outcomes from `paired.jsonl`; aggregate comparison fields are not used as a numerical source. This preserves task-level pairing and avoids crediting a plausible textual answer when the environment state is wrong.

#### 4.5 Preregistered inference and stopping

H1 compares MRPO with GRPO, H2 with no-minimax, and H3 with no-fission on the primary suite. A hypothesis passes only if all three seed-level paired differences are positive and the pooled task-paired bootstrap 95% confidence interval excludes zero. Bootstrap intervals use 2,000 resamples and random seed 7. The held-out L1 suite is reported separately and cannot rescue a failed primary hypothesis.

A regression gate examines the five legacy and two fault-v2 suites relative to r10. A confidence interval entirely below zero vetoes the round. The stopping rule is also frozen: if H1–H3 all fail, the result is recorded as strengthened negative evidence after estimator matching, and no fourth round is opened.

## 5 Results

### 5.1 Preregistered primary-suite decision

Table 1 reports verifier-confirmed successes on the frozen 32-episode primary suite. These values are read from the episode-paired evaluation artifacts; all comparisons use the same task identities within a seed.

None of the three preregistered hypotheses is supported. For H1 (MRPO versus GRPO), seed-level paired differences are  $-1, +1, 0$  and the pooled difference is  $0/96$ , with a 95% paired-bootstrap interval of  $[-0.0938, 0.0938]$ . For H2 (MRPO versus no-minimax), the differences are  $-5, -6, -3$  and the pooled difference is  $-14/96$ , with interval  $[-0.2604, -0.0312]$ . For H3 (MRPO versus no-fission), the differences are  $0, -3, +1$  and the pooled difference is  $-2/96$ , with interval  $[-0.1042, 0.0625]$ . Each hypothesis required a common positive direction at all seeds and a pooled interval excluding zero. H1–H3 therefore fail.

### 5.2 Regression gate and held-out behavior

The preregistered gate passes: every confidence interval associated with a nonzero regression warning includes zero. The null mechanism results are therefore not caused by one arm collapsing at a single seed. Relative to the r10 reference on the held-out suite, pooled paired intervals are  $[-0.0104, 0.1562]$  for MRPO,  $[0.0104, 0.1979]$  for GRPO,  $[0.0312, 0.1979]$  for no-fission, and  $[0.0521, 0.1979]$  for no-minimax. These secondary results show that formal RL can improve over the calibrated reference, but they do not support the specific MRPO mechanisms: the two arms without the claimed components have the clearest held-out gains.

## 6 Analysis

**Estimator matching changes the interpretation.** Failure-prefix fission can appear beneficial when it also creates more samples or changes the advantage group. After matching those quantities, MRPO minus no-fission changes sign across seeds and its pooled interval contains zero. In this regime, branching does not induce an identifiable behavioral improvement beyond the estimator and data-allocation changes that normally accompany it.

**Minimax weighting is the stable negative result.** No-minimax is better at every seed by 5, 6, and 3 primary-suite episodes. The pooled paired interval for MRPO minus no-minimax excludes zero below zero. A plausible explanation is that the calibrated policy already sits near a budget-dependent phase boundary: upweighting weak fault groups spends limited updates on sparse, high-variance gradients and moves probability mass away from broadly useful behavior. This explanation is mechanistic but not itself causally identified; the empirical conclusion is narrower and stronger: minimax weighting is harmful under the frozen protocol.

**Budget calibration is part of the result.** Pilot calibration prevented both a near-zero floor and a saturated ceiling. Even so, 32 binary episodes per seed yield wide intervals for small effects. The protocol can reliably reject the large positive mechanism story and can detect the larger minimax penalty, but it cannot rule out modest positive or negative effects for fission. Additional rounds chosen after seeing these results would weaken the preregistration; we therefore stop after the planned three seeds.

**Relation to Fission-GRPO.** Fission-GRPO is a full recovery-training system with its own simulator, diagnostic feedback, recovery-instance construction, and benchmark mix [21]. Our no-fission contrast is not an official reproduction of that system. It tests whether the broader branching idea retains a marginal advantage in our typed executable environments when its rollout estimator is controlled. The negative result limits transfer of the mechanism claim; it does not negate the reported end-to-end Fission-GRPO results.

## 7 Limitations and Broader Impact

The evidence comes from one model family and scale (Qwen3.5-9B with LoRA), a controlled fault taxonomy, three preregistered seeds, and small frozen binary evaluation suites. The paired design improves sensitivity but does not make small effects identifiable. Our structured diagnostics and exact state verifiers are stronger than signals available from many production APIs.

We planned a zero-touch transfer evaluation after freezing all adapters. ToolBench-X technically fits the tool interface but its dataset terms require prior permission for copying or modification. ToolMaze exposes no clearly applicable code or dataset license. We did not download or adapt either dataset and report no external transfer score. This legal and evidentiary boundary matters: controlled-environment findings have not been externally validated.

Tool-recovery training can increase agent autonomy. Deployment should retain authorization boundaries, idempotency controls, audit logs, and human confirmation for irreversible actions. Negative findings should likewise not be generalized into a claim that recovery training never works; they show that component benefits must be demonstrated under matched estimators and explicitly accounted budgets.

## References

- [1] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan.  $\tau^2$ -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025. URL <https://arxiv.org/abs/2506.07982>.
- [2] Jiajun Chai, Guojun Yin, Zekun Xu, et al. Rlfactory: A plug-and-play reinforcement learning post-training framework for llm multi-turn tool-use. *arXiv preprint arXiv:2509.06980*, 2025. URL <https://arxiv.org/abs/2509.06980>.

- [3] Yuxin Chen, Xiaodong Cai, Junfeng Fang, et al. Learning to act under noise: Enhancing agent robustness via noisy environments. *arXiv preprint arXiv:2605.27209*, 2026. URL <https://arxiv.org/abs/2605.27209>.
- [4] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [5] Yuelin Hu, Zhenbo Yu, Zhengxue Cheng, Wei Liu, and Li Song. Maximizing rollout informativeness under a fixed budget: A submodular view of tree search for tool-use agentic reinforcement learning. *arXiv preprint arXiv:2605.05262*, 2026. URL <https://arxiv.org/abs/2605.05262>.
- [6] Dongfu Jiang, Yi Lu, Zhuofeng Li, et al. Verltool: Towards holistic agentic reinforcement learning with tool use. *arXiv preprint arXiv:2509.01055*, 2025. URL <https://arxiv.org/abs/2509.01055>.
- [7] Song-Lin Lv, Weiming Wu, Rui Zhu, Zi-Jian Cheng, and Lan-Zhe Guo. Can agents generalize to the open world? unveiling the fragility of static training in tool use. *arXiv preprint arXiv:2607.01084*, 2026. URL <https://arxiv.org/abs/2607.01084>.
- [8] Jeffery Opoku and David Banahene. Toolchain-crc: Conformal risk control for agentic ai under retrieval and tool-use drift. *arXiv preprint arXiv:2606.18467*, 2026. URL <https://arxiv.org/abs/2606.18467>.
- [9] Cheng Qian, Emre Can Acikgoz, Qi He, et al. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025. URL <https://arxiv.org/abs/2504.13958>.
- [10] Yujia Qin, Shihao Liang, Yining Ye, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2307.16789>.
- [11] Ella Rabinovich and Ateret Anaby-Tavor. On the robustness of agentic function calling. *arXiv preprint arXiv:2504.00914*, 2025. URL <https://arxiv.org/abs/2504.00914>.
- [12] Shiori Sagawa, Pang Wei Koh, Tatsunori B. Hashimoto, and Percy Liang. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=ryxGuJrFvS>.
- [13] Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- [14] Yang Tian, Zhengpeng Shi, Yu Zhou, and Bo Zhao. Beyond function calling: Benchmarking tool-using agents under tool-environment unreliability. *arXiv preprint arXiv:2606.25819*, 2026. URL <https://arxiv.org/abs/2606.25819>.
- [15] Sri Vatsa Vuddanti and Satwik Kumar Chittiprolu. Recoverability has a law: The err measure for tool-augmented agents. *arXiv preprint arXiv:2601.22352*, 2026. URL <https://arxiv.org/abs/2601.22352>.
- [16] Ruipeng Wang, Yuxin Chen, Yukai Wang, et al. Agentnoisebench: Benchmarking robustness of tool-using llm agents under noisy condition. *arXiv preprint arXiv:2602.11348*, 2026. URL <https://arxiv.org/abs/2602.11348>.
- [17] Zhiheng Xi, Jixuan Huang, Chenyang Liao, et al. Agentgym-rl: Training llm agents for long-horizon decision making through multi-turn reinforcement learning. *arXiv preprint arXiv:2509.08755*, 2025. URL <https://arxiv.org/abs/2509.08755>.
- [18] Minrui Xu, Zilin Wang, Mengyi Deng, et al. Envfactory: Scaling tool-use agents via executable environments synthesis and robust rl. *arXiv preprint arXiv:2605.18703*, 2026. URL <https://arxiv.org/abs/2605.18703>.

- [19] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [20] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan.  $\tau$ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024. URL <https://arxiv.org/abs/2406.12045>.
- [21] Zhiwei Zhang, Fei Zhao, Rui Wang, Zezhong Wang, Bin Liang, Jiakang Wang, Yao Hu, Shaosheng Cao, and Kam-Fai Wong. Robust tool use via fission-grpo: Learning to recover from execution errors. *arXiv preprint arXiv:2601.15625*, 2026. URL <https://arxiv.org/abs/2601.15625>.
- [22] Xiaolin Zhou, Aojie Yuan, Zheng Luo, et al. When simulation lies: A sim-to-real benchmark and domain-randomized rl recipe for tool-use agents. *arXiv preprint arXiv:2605.11928*, 2026. URL <https://arxiv.org/abs/2605.11928>.